COMPILERS · TEAM 6

Parser + SDT

A syntactic and semantic approach to code validation.

Arroyo Onofre Leonardo
423054589

Camacho Ignacio Violeta
319061345

Castañeda Luna Valeria
320047822

Martínez Pérez Diego Alexander
423020441

Ruiz Ramírez Santiago
320263635

Saucedo Serralde Alejandra Melissa
320284858

SEMESTER 2026-II · MEXICO CITY, APRIL 2026

01 · THE PROBLEM

Source Code, Without a Parser.

```
input.txt

int result = (3 + 5) * 2 ;

is it valid?  what type is it?  is it declared?

lexer →  keyword  identifier  operator
                 constant  punctuation
```

1 **No structure**
A flat list of tokens says nothing about order or hierarchy.

2 **No types**
We don't know if int = "hello" is legal.

3 **No context**
We can't detect identifiers used before being declared.

A front-end compiler, from scratch, in Python.

No parsing libraries. Every decision, visible and traceable.

Parser LL(1)

Top-down, explicit table and stack. Zero backtracking.

Semantic SDT

Symbol table, type inference, and assignment compatibility.

3 visual trees

Parse tree, verified parse tree, and AST, exported as PNG.

How It Works

input.txt
source code

Lexer
enriched tokens

Parser LL(1)
parse tree

SDT
validation + types

AST
final tree

OUTPUTS PER RUN

first_follow.txt · parsing_table.txt · parse_trace.txt

parse_tree.png · verified_parse_tree.png · abstract_tree.png

Four concepts. Sixty seconds.

LL(1)

Top-down, 1 lookahead token

Reads left to right. Decides the production by looking at the next single token.

FIRST / FOLLOW

The automaton's keys

FIRST(α) = first terminals. FOLLOW(A) = what can come after. They build the LL(1) table.

SDT

Semantic rules over the tree

Symbol table, redeclaration and undeclared identifier detection, type propagation.

AST

The tree that actually matters

Compact version of the parse tree. Only semantically relevant nodes, ready for the next phase.

The Grammar

9 productions, 4 types, fully LL(1) compatible.

grammar.txt

```
S → DECLARATION
DECLARATION → TYPE id DECLARATION'
DECLARATION' → = EXPRESSION ; | ;
TYPE → int | float | char | double
EXPRESSION → TERM EXPRESSION'
EXPRESSION' → + TERM EXPRESSION' | -
TERM EXPRESSION' | ε
TERM → FACTOR TERM'
TERM' → * FACTOR TERM' | / FACTOR
TERM' | ε
FACTOR → id | const | ( EXPRESSION )
```

LL(1) PARSER · COMPONENTS

✓ **Parsing table**

FIRST/FOLLOW build the lookup grid.

✓ **Stack**

Holds expected non-terminals.

✓ **Cursor**

Reads tokens left to right.

✓ **Productions**

Predict, expand, or match.

Four cases. Three verdicts.

TEST 1

OK

```
int a = 5;
```

Valid declaration.
Parse + SDT verify.

TEST 2

OK

```
int result = (3 + 5) * 2;
```

Arithmetic with precedence.
Inferred type: int.

TEST 3

SEMANTIC ERROR

```
int x = 8.1;
```

Parse OK; SDT detects
decimal assigned to int.

TEST 4

SYNTAX ERROR

```
int x = ;
```

Parser fails:
expression expected.

The Three Trees

Three views of the same input. Each one strips away noise.

Parse Tree

parse_tree.png

Every production applied, including auxiliary nodes and ε.

Verified Tree

verified_parse_tree.png

Symbol table, types, and assignment compatibility annotated.

AST

abstract_tree.png

Compact tree, only semantically meaningful nodes.

Symbol table · Type inference · Assignment compatibility

Pricing

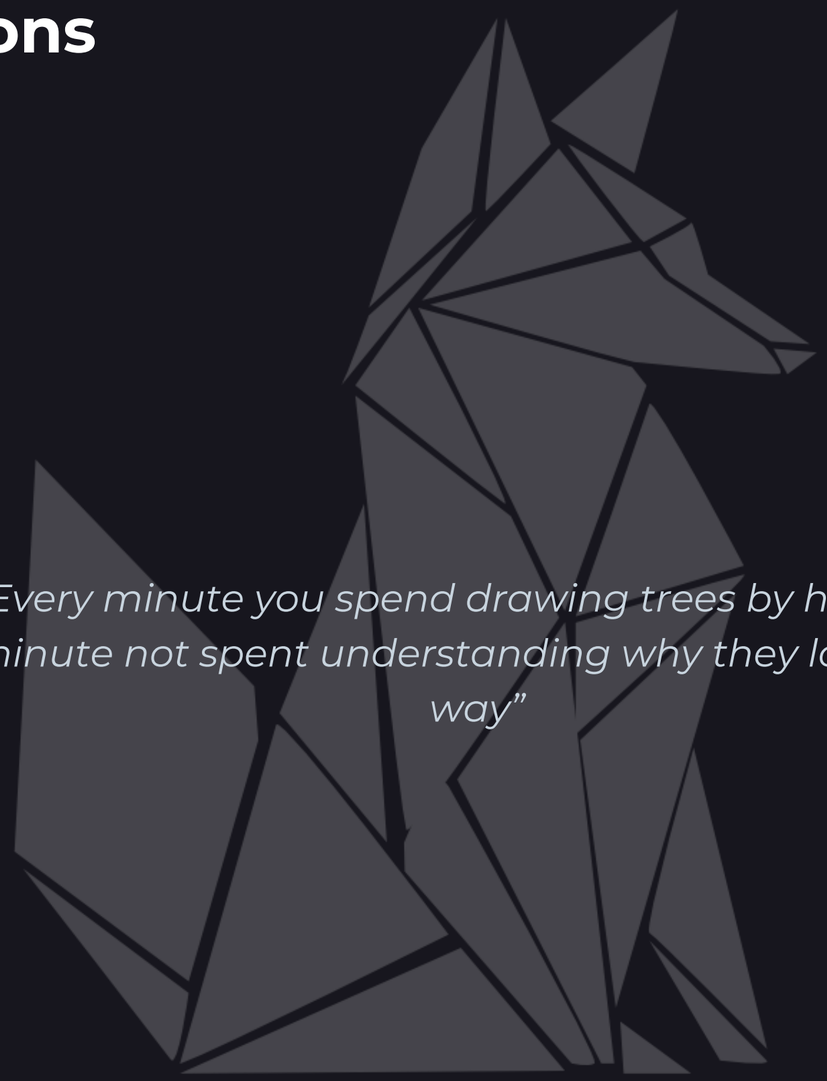
One price, no subscriptions

Save up to:

98.89%

of you're time

1. **3 parse full tree**
2. **Step-by-step trace**
3. **Semantic error detection**



"Every minute you spend drawing trees by hand is a minute not spent understanding why they look that way"

Standard Edition
For all public

From

$49 USD

Educational Edition
For students and professors

From

$ 29 USD

Try it first
7 days - Full access

From

$ 0 USD

Questions?

Thank You